

Why Solve It Twice? Hierarchical Accumulation of Skills for Transfer-Efficient ML Engineering

Anonymous Authors¹

Abstract

ML engineering agents waste compute rediscovering known techniques because every competition is a cold start. We present HASTE, a hierarchical multi-agent system that organizes cross-competition knowledge into three scope tiers (global, domain, and competition-specific), each coupled to a matching agent level. An orchestrator coordinates domain specialists and promotes learning between tiers via LLM-driven abstraction. A controlled ablation provides evidence for scoped loading: holding a 159-skill inventory constant across 8 competitions, tiered loading achieves a 100% medal rate while flat loading reaches only 62.5%, the same medal rate as loading no skills, and consumes $2\times$ the output tokens. On the full MLE-Bench Lite benchmark (22 Kaggle competitions), HASTE reaches a medal rate of 77.3% using Claude Sonnet 4.6 at 12h per competition. In a cold-start run, the system begins with no accumulated skills. In warm-start runs, it reloads skills learned from earlier competitions, using only global and domain-level skills for transfer across competitions. Warm starts use 52% fewer refinement iterations, and the fraction of proposed changes kept by the agent rises from 42% at low inventory to 85% once 50+ skills are available. These results suggest that better knowledge organization can partly substitute for model strength and compute budget in ML-engineering agents.

1. Introduction

Why solve the same problem twice? Current ML engineering agents do exactly this. MLE-Bench evaluates agents on 75 Kaggle competitions independently (Chan et al., 2025),

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

and agents treat them independently too, resetting all state between tasks. Techniques that proved effective in one competition must be rediscovered from scratch in the next similar one. This redundancy is the cost of treating every competition as a cold start: many top-performing agents rely on frontier models, longer budgets, or both to compensate for repeated exploration.

Recent work has explored cross-task transfer (Grosnit et al., 2024; Zhang et al., 2024; Zhao et al., 2023; Wang et al., 2023; Hu et al., 2024), but stores knowledge in flat pools or by memory type. Agents with flat memory still lack the *organization* needed to load the right knowledge for the right task: everything goes into one context window, diluting the signal. Transfer helps only when the agent can select the right prior for the current task. The organization of accumulated knowledge therefore determines whether the agent spends its limited budget on useful exploration or on re-deriving known facts. We provide direct evidence that this distinction is consequential. In our controlled ablation, flat loading performs no better than loading no skills, whereas scoped hierarchical loading medals on every task.

HASTE (Hierarchical Accumulation of Skills for Transfer-Efficient ML Engineering) organizes accumulated skills into three scope tiers: global skills that apply across ML tasks, domain skills for tabular, vision, NLP, or audio tasks, and competition skills that remain tied to one dataset. An orchestrator coordinates domain specialists for tabular, vision, and NLP, each loading only the skills relevant to its scope. Between competitions, the orchestrator promotes learning upward via LLM-driven abstraction. This scoped loading means each agent sees only what is relevant, keeping prompts focused and iterations productive. The loaded skills act as a structured prior over the next code change. That prior keeps the per-task search narrow enough for a linear refinement loop to work: after two failed improvements the loop moves from broad exploration to optimization or fine-tuning, and any change that hurts validation score is reverted before the next step.

Across a multi-phase MLE-Bench Lite campaign, HASTE reaches the top public performance band using a non-frontier model and a 12h budget. Skill accumulation improves medal rate, reduces the number of refinement itera-

tions needed, and raises the fraction of proposed changes that are worth keeping. A fixed-inventory ablation shows that accumulation needs scoped organization: tiered loading beats both flat and empty loading, while flat matches empty at much higher token cost.

Contributions. The paper contributes a 3-tier hierarchical skill store with LLM-driven promotion between global, domain, and competition tiers, coupled to an orchestrator and matching domain specialists. It evaluates scoped loading directly with a fixed-inventory ablation, showing that hierarchy matters beyond skill volume: the same 159 skills help when loaded by scope but not when dumped into one flat prompt. To our knowledge, HASTE is the first MLE-Bench agent to organize reusable skills by cross-competition scope and evaluate scoped loading against flat and empty loading under a fixed inventory. The empirical study shows that this organization improves efficiency on MLE-Bench Lite, allowing a non-frontier model under a 12h budget to reach the same public performance band as stronger-model systems: a cold run with no accumulated skills achieves 40.9% medal rate, while reloading global and domain skills lifts the same system to 77.3%, flipping 8 of 13 previously failed competitions to medal. The study characterizes the resulting 159-entry plain-text skill inventory across 3 tiers and 4 domains.

2. Related Work

MLE-Bench agents and search strategies. A first family of MLE-Bench agents advances the *search* axis. AIDE (Jiang et al., 2025) runs greedy tree search; MLE-STAR (Nam et al., 2025) performs ablation-guided targeted refinement with web-retrieved priors; AIRA-Dojo (Toledo et al., 2025) formalizes MLE agents as search-policy $\times$ operator-set; R&D-Agent (Yang et al., 2025) separates a Researcher from a Developer. Population-based evolutionary search appears in LoongFlow (Wan et al., 2025) and FM Agent (Li et al., 2025), whereas budget-aware or graph-augmented Monte Carlo search appears in MARS (Chen et al., 2026) and ML-Master (Liu et al., 2025). HASTE contributes to the *knowledge* axis, which is orthogonal to search strategy and can be combined with any of these frameworks.

Cross-task knowledge transfer in LLM agents. Several systems accumulate experience across tasks but store it as a flat pool. Voyager (Wang et al., 2023) maintains a code skill library indexed by embedding similarity in Minecraft; ExpeL (Zhao et al., 2023) extracts natural-language insights via ADD/EDIT/UPVOTE/DOWNVOTE on a flat vector store; Reflexion (Shinn et al., 2023) pioneered verbal self-reflection within a single task; ICAL (Sarch et al., 2024) extends reflection to vision-language agents with a four-component knowledge structure. For data science

specifically, Agent K (Grosnit et al., 2024) maintains persistent intrinsic state summarizing past episodes across competitions within an experiential-learning formalism; ML-Copilot (Zhang et al., 2024) retrieves related benchmarks via text embeddings and distilled knowledge; DS-Agent (Guo et al., 2024) combines embedding-ranked human-insight cases with iterative case-based reasoning; MLZero (Fang et al., 2025) separates semantic library knowledge from episodic execution traces; ADAS (Hu et al., 2024) evolves agents via Meta Agent Search over an archive of coded designs. SkillRL (Xia et al., 2026) reports a 25% performance collapse when distilled skills are replaced with raw trajectories, motivating aggressive compression.

Table 1. Cross-task knowledge mechanisms in ML engineering agents. $\checkmark$ = present, $\times$ = absent, $\sim$ = partial.

System	Organization	Cross-Task	Hierarchy	Promotion	MLE-Bench
AIDE	None	$\times$	$\times$	$\times$	$\checkmark$
MLE-STAR	Flat (external)	$\times$	$\times$	$\times$	$\checkmark$
R&D-Agent	Flat (per-task)	$\sim$	$\times$	$\times$	$\checkmark$
PiEvolve ¹	Flat (per-task)	$\times$	$\times$	$\times$	$\checkmark$
MLEvolve	Flat (per-task)	$\sim$	$\times$	$\times$	$\checkmark$
MARS	Flat (cross-branch)	$\sim$	$\times$	$\times$	$\checkmark$
Agent K v1.0	Flat (shared)	$\checkmark$	$\times$	$\times$	$\times$
MLCopilot	Flat (vector DB)	$\checkmark$	$\times$	$\times$	$\times$
ExpeL	Flat (vector store)	$\checkmark$	$\times$	$\times$	$\times$
MLZero	Semantic + Episodic	$\checkmark$	$\times$	$\times$	$\sim$
Voyager	Flat (embedding)	$\checkmark$	$\sim$	$\times$	$\times$
ADAS	Flat (archive)	$\checkmark$	$\times$	$\times$	$\times$
HASTE (ours)	3-tier hierarchy	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

Table 1 compares these systems by organization, cross-task scope, hierarchy, promotion, and MLE-Bench evaluation; HASTE is the only one that organizes reusable knowledge by *scope of applicability*.

Hierarchical organization in agent systems. Hierarchy is well-motivated outside ML engineering. The options framework (Sutton et al., 1999) introduced temporally extended actions in RL, building on feudal RL (Dayan & Hinton, 1993) where a manager sets sub-goals for a worker; feudal networks (Vezhnevets et al., 2017) formalized this with different temporal resolutions. For LLM agents, GITM (Zhu et al., 2023) uses an explicit three-tier decomposition for Minecraft. It is the closest published precedent for a 3-tier organization, although its tiers are within-task layers rather than cross-task scope layers. CoALA (Sumers et al., 2024) classifies agent memory by type, and Talebirad et al. (2026) formalize how bounded-capacity agents benefit from multi-level knowledge organization. HASTE brings this principle to ML engineering with empirical validation. Its 3-tier skill store and orchestrator-specialist hierarchy mirror a feudal architecture in which managers at different levels decide what knowledge a subordinate executor should see, while the stored knowledge itself is organized by *scope of applicability*.

¹<https://github.com/FractalAIRResearchLabs/PiEvolve>

Automated pipeline search and meta-learning. Earlier AutoML systems reduce the cost of building a pipeline for each new dataset by searching within predefined model and hyperparameter spaces. Predefined-space systems such as AutoGluon (Erickson et al., 2020), TPOT (Olson & Moore, 2016), Auto-WEKA (Thornton et al., 2013), and Auto-Sklearn (Feurer et al., 2015; 2022) search hand-engineered configuration grids. Meta-learning extensions (e.g., Auto-Sklearn warm-starting) accumulate cross-dataset experience as numerical meta-features and predict configuration vectors on new tasks. These systems are effective within their predefined space but cannot incorporate qualitative insights (“tokenization choice X breaks on dataset Y because of Z”). AgentHPO (Liu et al., 2024) tunes hyper-parameters; CAAFE (Hollmann et al., 2023) generates feature-engineering code; EvoPrompting (Chen et al., 2023) searches architectures; AutoML-GPT (Zhang et al., 2023) orchestrates training from model and data cards. Across this group, accumulated priors remain flat or implicit; HASTE splits them into applicability tiers. HASTE keeps the same goal of reducing per-task search cost, but the search takes place in unbounded Python code rather than a fixed configuration grid. It makes that larger space tractable by reusing plain-text lessons extracted from prior runs and loading only the lessons whose scope matches the current task.

3. Method

Figure 1 shows the two hierarchies that co-evolve during a multi-competition run. The *skill hierarchy* has global, domain, and competition tiers. The *agent hierarchy* has an orchestrator and three domain specialists for tabular, NLP, and vision tasks. Scoped context loading connects them: each agent receives only the skill tiers matched to its scope. The search space remains unbounded Python code; we restrict only the *distribution* over that space using accumulated structured priors.

3.1. Skill Hierarchy

The skill store is a plain-text filesystem of markdown files with YAML frontmatter, organized into three tiers by *scope of applicability*. **Global** has 5 entries and is loaded by every specialist. **Domain** has 12 NLP, 19 tabular, and 15 vision entries, loaded only by the matching specialist. **Competition** has 108 entries across 21 directories, loaded only on re-runs of that competition.

Within each tier, HASTE distinguishes three kinds of entries, written into different prompt slots at different stages. *Technique* entries record what worked or failed and feed proposal prompts, for example “target encoding helps high-cardinality categoricals in tree models.” *Commitment priors* record which design choices have high cross-task vari-

ance and feed the prototype screen, telling the agent which choices require evidence before commitment. *Refinement hints* record which knobs to tune per model family and feed the optimizing and fine-tuning stages. Other systems collapse these into one “lessons” bag; we separate them because they answer different questions and enter the loop at different points.

Loading is deliberately simple: read the relevant directories and concatenate them. No embedding index is needed at the current scale of roughly 159 entries, with 10 to 60 loaded per agent. A character cap of 2000 in the prototype prompt and 4000 in refinement limits dilution if the inventory grows. The decision to avoid embedding-only retrieval follows recent results on the theoretical limits of single-vector embeddings for multi-field conditional retrieval (Weller et al., 2026). Most skills come from an LLM reflection step at the end of each competition (Figure 5), in the spirit of Reflexion (Shinn et al., 2023) and ExpeL (Zhao et al., 2023), with paired success and failure analysis so that failure modes are recorded explicitly. Two skill types are additionally extracted *algorithmically* by a Prior Extractor. Commitment priors come from score variance across prototype model options; high variance implies a decision-relevant choice. Refinement hints come from per-knob deltas across refinement history, identifying which changes had the highest acceptance rate. These structured priors are mined from logs; to our knowledge, prior text-memory agents rely primarily on LLM summarization for this kind of memory.

3.2. Agent Hierarchy and Execution Loop

Orchestrator. The orchestrator delegates experiment execution. Its responsibilities are: (i) *domain classification*, assigning each competition to a domain via manual tags for 100+ MLE-Bench competitions with a heuristic fallback; (ii) *round scheduling*, picking seeds first (one per domain for cold start) and then assigning remaining competitions to specialists; and (iii) *skill promotion*, evaluating new learnings via an LLM after each round (Figure 6). Promotion decides for each learning: *skip* (already covered), *competition* (too specific), *domain* (abstract and promote up one tier), *global* (universally useful), or *conflict* (contradicts an existing skill). When two learnings conflict, both are kept and annotated with conditions. For example, one skill records that ensembling helps when correlation is below 0.95, but hurts when a weaker member pulls down a stronger one. The store grows through *abstraction* and stays interpretable plain text on the filesystem. The top-level multi-competition loop is given in Algorithm 1 (Appendix A.1).

Specialist. Given a competition t and the scope-loaded skills supplied by the orchestrator, the specialist runs a five-

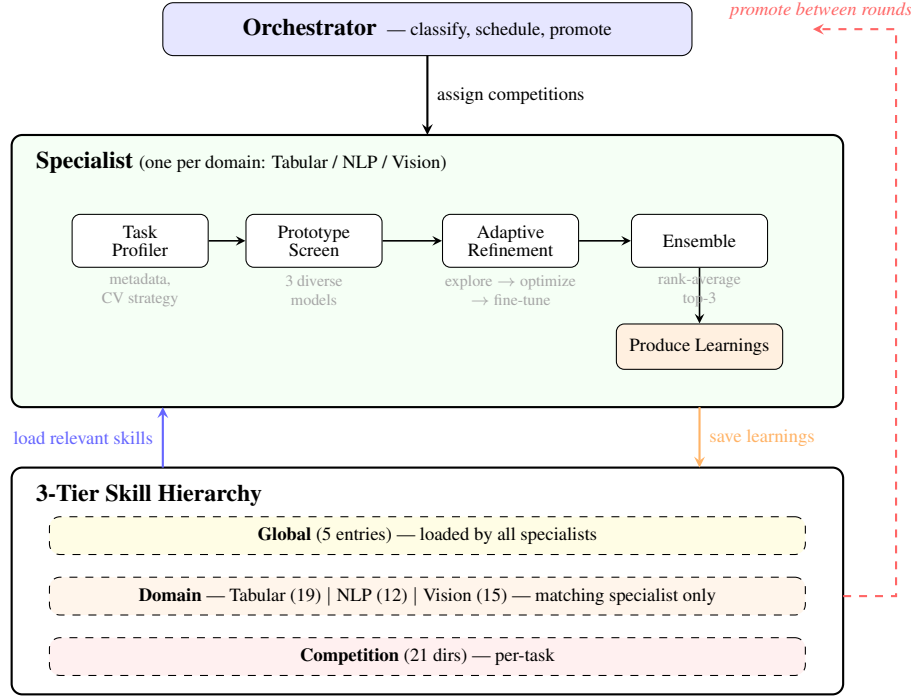


Figure 1. HASTE architecture. The Orchestrator assigns competitions to domain Specialists. Each Specialist loads relevant skills, executes the pipeline (profile $\rightarrow$ prototype $\rightarrow$ refine $\rightarrow$ ensemble), and produces learnings. Between rounds, the Orchestrator promotes generalizable learnings upward through the hierarchy via LLM-driven abstraction.

stage per-competition pipeline. Appendix A.2 gives the full pseudocode in Algorithm 2. **(1) Task profiling:** parse metadata, profile the dataset, select a CV strategy, and probe GPU/CPU/RAM; no model execution. **(2) Prototype screen** (Figure 3): the LLM proposes three fundamentally diverse approaches and executes each on a single validation fold; up to a $2.7\times$ score spread between best and worst justifies this hedge against committing to a suboptimal foundation. **(3) Adaptive refinement** on the prototype winner with budget $N=20$ and on the runner-up with $N=6$ via ADAPTIVEREFINE (Appendix A.3), a linear loop across three tiers (Exploring, Optimizing, Fine-tuning) with auto-escalation on two consecutive non-improvements, stagnation exit at Fine-tuning, and revert-on-regression on every step (Figure 4); the runner-up branch is a second hedge, and in 3 of our 25 main-benchmark runs, the runner-up’s refined score beat the winner’s. **(4) Ensemble:** rank-average the top-3 checkpoints across both branches; accept the ensemble only if it beats the best single member. **(5) Produce learnings** (Figure 5): the LLM reflects on the full experiment history and emits 2 to 5 plain-text learnings, each with a proposed tier, which flow back to Algorithm 1 for promotion. A 6-mode failure taxonomy guides refinement diagnosis as prompt content: UNDERFITTING, OVERFITTING, FEATURE_GAP, NOISE_CEILING, DISTRIBUTION_MISMATCH, and DIMINISHING_RETURNS. Tiered history compression keeps the prompt under 20 lines even

at 50+ iterations.

This linear refinement loop is intentionally simpler than the trees, evolutionary populations, or MCTS used by other systems (Jiang et al., 2025; Li et al., 2025; Chen et al., 2026; Liu et al., 2025; Wan et al., 2025). We found linear refinement sufficient in this setting; the priors loaded by the orchestrator plausibly collapse the branching factor enough for linear search to match tree or evolutionary alternatives, but a controlled comparison at fixed knowledge condition is future work. The ablation in §4.5 shows the priors carry substantive weight at fixed search strategy: performance drops sharply when no skills are loaded.

Executor. The executor is a pluggable backend; the agent has Read, Write, Edit, Bash, Glob, and Grep tools, a 3-attempt retry per step with diagnostic feedback between attempts, and validates submissions against `sample_submission.csv` (column names, row count, no NaN, dtype) before scoring. For cost efficiency, all experiments in this paper use the CLI backend (`claude -p subprocess`) billed via a fixed Claude Code subscription.

4. Experiments

4.1. Setup

We evaluate on MLE-Bench Lite (Chan et al., 2025), the 22-competition subset used for cross-agent comparison. Each competition runs Claude Sonnet 4.6 via the CLI backend on a SLURM node with 24 CPUs, 128 GB RAM, and 1 NVIDIA L40S 48 GB GPU. The wall-clock budget is 12h, half the dominant 24h budget on the leaderboard, with 30 iterations on the main benchmark and 11 on the ablation. The search space is unbounded Python code; evaluation uses the official Kaggle metric per competition, scored against held-out test sets via the MLE-Bench grader. No test labels are used in the iteration loop. Appendix B includes representative prompts, and Appendix E shows representative skills; we plan to release the full codebase upon acceptance.

4.2. Main Results

Table 2. MLE-Bench Lite reference points.

Agent	LLM	Medal %	Time
Famou-Agent 2.0	Gemini-3-Pro	80.3	24h
MLEvolve	Gemini-3-Pro	80.3	12h
PiEvolve	Gemini-3-Pro	80.3	24h
CAIR MARS+	Gemini-3-Pro	78.8	24h
HASTE (ours)	Claude Sonnet 4.6	77.3	12h
AIBuildAI	Claude Opus 4.6	77.3	24h
Famou-Agent 2.0	Gemini-2.5-Pro	75.8	24h
ML-Master 2.0	Deepseek-V3.2	75.8	24h
CAIR MARS	Gemini-3-Pro	74.2	24h
PiEvolve (12h)	Gemini-3-Pro	74.2	12h

Across the multi-phase accumulation campaign, HASTE’s medal rate on MLE-Bench Lite is 77.3% (17 of 22), with 10 gold, 2 silver, and 5 bronze. The above-median rate is 86.4%. The per-competition table is in Appendix C.

Table 2 places HASTE among public MLE-Bench Lite results. At 77.3%, HASTE reaches the same performance band as leading public MLE-Bench Lite agents while using a non-frontier model and a 12h budget. Two features distinguish it from the rest of the top-tier agents. HASTE is the only non-frontier-model agent at or above 77%: every other agent in this band uses Gemini-3-Pro-Preview, Claude Opus 4.6, or a model ensemble. HASTE is also one of only two top-band agents running at 12h; all others use the standard 24h budget. We note that public leaderboard numbers carry per-task statistical noise (paper-reported SD ≈ 4.4) and that a multi-seed confidence interval is still pending; differences of less than 5 points on Lite fall within that noise, and the headline 77.3% is a campaign result rather than a multi-seed estimate (§5.3).

A cold-start single-pass of HASTE achieves only 40.9%,

which would place it below every agent on Table 2; tiered skill accumulation adds 36.4 percentage points to the same system, lifting it into the leading public performance band.

4.3. Evidence for Skill Transfer

Skill accumulation lifts medal rate from 40.9% (9 medals) in a single-pass cold run to 77.3% (17 medals) when later phases reload accumulated skills. A cold run starts without prior competition experience. A warm run starts with skills learned earlier in the campaign. For the warm transfer evaluation, HASTE reloads global and domain skills only. Competition-specific skills are not loaded, so the result measures transfer from reusable knowledge rather than leakage from a previous attempt on the same dataset.

The improvement shows up in three ways. First, warm-start runs need fewer refinement iterations to reach their best score: 7.8 on average compared with 16.3 for cold-start runs, a 52% reduction. Second, the agent keeps more of its own proposed edits as the skill inventory grows. We measure hit rate as the fraction of attempted changes that improve validation score and are kept rather than reverted. When the inventory has 0 to 15 skills, the hit rate is 42%; once 50+ skills are available, it rises to 85%. Third, of the 13 competitions that failed the cold run, 8 flipped to medal in the warm run, while the 9 competitions that already medaled cold were skipped in the rerun phase (Appendix D). The store grew from 5 entries at cold start to 72 by phase 12, and each promotion step used LLM abstraction to keep the entries readable as plain text.

4.4. Phase 0, Refinement, and What the Agent Learned

Refinement beats the prototype winner in 92% of runs (23 of 25), with an average gain of +0.045 in the competition’s native metric; in 3 competitions, the runner-up’s refined score beat the winner’s, and the two cases where refinement underperformed both occurred in early phases with fewer skills. The final 159-entry skill store breaks down as 5 global, 15 vision, 12 NLP, 19 tabular, and 108 competition entries, all traceable to specific runs and including both successes and failures. For example, the global skill “ensembling a strong model with a weaker model can degrade performance” was learned from dogs-vs-cats and later prevented the same mistake in two vision competitions; “DeBERTa-v3-base is the strongest text-classification start under a 12h budget” was learned from detecting-insults and applied in jigsaw-toxic and spooky-author. Appendix E lists representative skills with source competitions and estimated iteration savings.

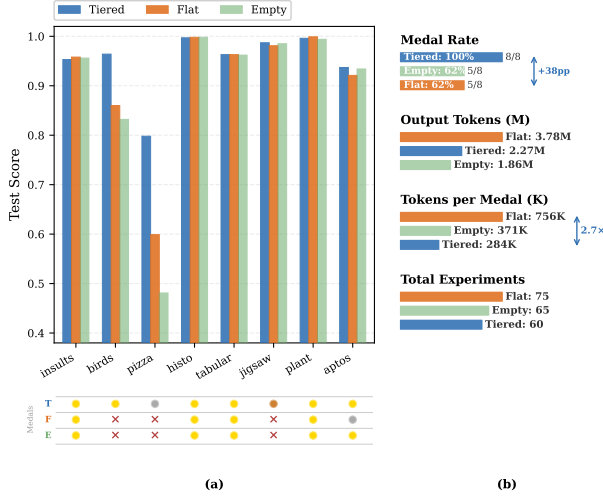


Figure 2. **Controlled ablation in Section 4.5.** All conditions use the same 159-skill inventory, model, pipeline, and budget. Tiered loading medals on all 8 competitions; flat and empty both medal on 5 of 8.

4.5. Ablation: Tiered vs. Flat vs. Empty Skill Loading

We test the loading function directly while holding skill *inventory* constant, varying only skill *organization*. We run 8 competitions spanning NLP, vision, tabular, and audio domains under three conditions, with the same model (Claude Sonnet 4.6), pipeline, 11-iteration budget, and 159-skill inventory frozen at one point in the benchmark campaign. The conditions differ only in *which* skills enter context: **Tiered** loads global, domain-matched, and competition-specific skills only (10 to 60 entries per competition); **Flat** loads all 159 skills (145K characters), the “dump everything into the prompt” approach; **Empty** loads no skills.

The difference between loading functions is large despite the fixed inventory. Tiered loading achieves 100% medal rate (6 gold, 1 silver, 1 bronze), flat 62.5% (4 gold, 1 silver, 3 no-medal), and empty 62.5% (5 gold, 3 no-medal); mean test scores follow the same ordering ($0.949 > 0.910 > 0.893$). Figure 2 shows the per-competition breakdown; the full table is in Appendix G.

Because the ablation is small, we treat the statistical tests as supporting evidence rather than a definitive test. With $N = 8$ and a single seed, formal tests are underpowered, but the qualitative pattern holds across measures. A paired bootstrap on per-competition score differences (tiered minus flat) gives a 95% CI of $[+0.001, +0.093]$ around a mean of $+0.040$, which just excludes zero. The corresponding one-sided Wilcoxon signed-rank tests yield $p = 0.11$ for tiered $>$ flat, $p = 0.08$ for tiered $>$ empty (both paired across the same 8 competitions), and Fisher’s exact test on medal counts gives $p = 0.10$ for tiered (8/8) vs. flat or empty (5/8). The tests fall short of standard significance

at $N = 8$; we report them transparently and emphasize the consistent direction of effect across all three measures. Multi-seed replication is the planned next step (§5.3).

The flat condition is especially informative because it tests whether more knowledge is enough. Flat skill loading leaves the medal rate unchanged relative to starting from scratch. Flat consumes 3.78M output tokens to empty’s 1.86M (tiered: 2.27M), doubling empty’s token cost for the same medal rate and a modestly higher mean score. The *tokens per medal* metric (total output tokens divided by medals won) makes the efficiency gap concrete: tiered spends 284K tokens per medal, flat 756K, and empty 371K (Figure 2b). Tiered is $2.7\times$ more token-efficient than flat per medal won. Flat runs the most experiments (75 vs. tiered’s 60 and empty’s 65) with a slightly higher execution success rate (87% vs. 83%), so the extra attempts appear poorly directed: more compute without medal-rate gains. Full resource breakdown is in Appendix F.

The aggregate result hides where the difference is largest. The gap focuses on harder or niche tasks, where competition-specific skills provide a known-good starting point and domain skills guide model selection. On *mlsp-2013-birds* (audio), tiered scores 0.964 (gold) vs. flat 0.860 and empty 0.832 (neither medal); on *random-acts-of-pizza*, tiered 0.798 (silver) vs. flat 0.599 and empty 0.481. The 4 easier competitions, detecting-insults, histopathologic, tabular-playground, and plant-pathology, show smaller differences and all three conditions medal.

From the logs, three mechanisms appear to limit flat loading. *Signal dilution*: relevant skills are buried among domain and competition-specific skills for unrelated competitions. *Context budget displacement*: at 145K characters, the flat skill dump crowds out the agent’s own reasoning and code analysis. *Overconfident model selection*: in the flat-jigsaw rerun, the agent repeatedly attempted DeBERTa-v3-large (an aggressive NLP recommendation that triggered OOM) while the empty agent used simpler models and scored higher (0.985 vs. 0.981).

Scope of the ablation. The flat condition couples three axes: organization, skill volume, and prompt-length budget ($\sim 145K$ vs. $\sim 25K$ characters). Because scoping is the mechanism by which the right 30 skills enter a 25K-character budget, a flat-but-character-capped condition that loads N random skills would test random subset selection, a different question. The supported claim is that, on this inventory, scoped loading outperforms both the flat full-load and empty-context baselines through the three mechanisms above; any broader claim about hierarchy reduces here to its role in enabling scoped loading at a controlled budget. The 159-skill inventory was itself accumulated under hierarchical promotion, so this ablation tests inventory transfer

under different loading functions; a fully flat pipeline end-to-end remains a separate condition.

5. Discussion

5.1. Why Hierarchy Makes Agents Faster

The efficiency gain depends on *scoped loading* in addition to accumulation, and the ablation supports this as the main mechanism. A flat skill store sends every specialist unrelated advice, including domain tricks for other modalities and competition-specific quirks from unrelated tasks. Tier scoping keeps the context small and matched to the current agent; the resource table shows the cost of failing to scope, with flat loading spending substantially more tokens without improving on empty’s medal rate.

5.2. Per-Domain Performance and Token Usage

Per-domain medal rates on the full benchmark are uneven: vision 80% (8/10), NLP 100% (6/6), tabular 40% (2/5), and the single audio competition (mlsp-2013-birds) reached gold. Remaining failures such as taxi-fare and dog-breed likely need approaches still absent from the skill store. Token usage scales sub-linearly with skill inventory: warm-start runs need fewer refinement iterations, so per-competition tokens drop as the store grows; per-condition totals are in Appendix F.

5.3. Limitations

Single-seed evaluation is the main limitation. This choice matches the efficiency positioning of the work, but multi-seed replication at the full 75-competition benchmark is the priority for follow-up runs with additional compute. The §4.5 ablation tests scoped loading while holding inventory fixed. Other engineering components, including the prototype screen, runner-up branch, rank-average ensemble, failure taxonomy, auto-escalation, and revert-on-regression, remain for an analogous component-wise study.

6. Conclusion

These results suggest that knowledge organization can partly substitute for model strength and compute budget in ML-engineering agents: HASTE reaches competitive MLE-Bench Lite performance with a non-frontier model under a shorter budget, and the fixed-inventory ablation points to scoped loading as the source of the gain over skill volume alone. Multi-seed replication, held-out evaluation, full 75-competition runs, and retrieval-augmented loading at scale are next.

Impact Statement

This work aims to improve the efficiency of ML engineering agents through reusable knowledge organization. Its broader impacts are those of ML engineering automation

generally; we do not foresee additional direct societal risks from the hierarchy mechanism itself.

References

- Chan, J. S., Chowdhury, N., Jaffe, O., et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *ICLR*, 2025. URL <https://arxiv.org/abs/2410.07095>.
- Chen, A., Dohan, D. M., and So, D. R. EvoPrompting: Language models for code-level neural architecture search. In *NeurIPS*, 2023.
- Chen, J., Mishra, B. D., Nam, J., Meng, R., Pfister, T., and Yoon, J. MARS: Modular agent with reflective search for automated AI research, 2026. URL <https://arxiv.org/abs/2602.02660>.
- Dayan, P. and Hinton, G. E. Feudal reinforcement learning. In *NeurIPS*, 1993.
- Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., and Smola, A. AutoGluon-tabular: Robust and accurate AutoML for structured data, 2020. URL <https://arxiv.org/abs/2003.06505>.
- Fang, H., Han, B., Erickson, N., et al. MLZero: A multi-agent system for end-to-end machine learning automation, 2025. URL <https://arxiv.org/abs/2505.13941>.
- Feurer, M., Klein, A., Eggenberger, K., Springenberg, J., Blum, M., and Hutter, F. Efficient and robust automated machine learning. In *NeurIPS*, 2015.
- Feurer, M., Eggenberger, K., Falkner, S., Lindauer, M., and Hutter, F. Auto-Sklearn 2.0: Hands-free AutoML via meta-learning. *JMLR*, 23, 2022.
- Grosnit, A., Maraval, A., Doran, J., et al. Kolb-based experiential learning for generalist agents with human-level kaggle data science performance, 2024. URL <https://arxiv.org/abs/2411.03562>.
- Guo, S., Deng, C., Wen, Y., Chen, H., Chang, Y., and Wang, J. Ds-agent: Automated data science by empowering large language models with case-based reasoning. In *ICML*, 2024. URL <https://arxiv.org/abs/2402.17453>.
- Hollmann, N., Müller, S., and Hutter, F. Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering. In *NeurIPS*, 2023. URL <https://arxiv.org/abs/2305.03403>.

- Hu, S., Lu, C., and Clune, J. Automated design of agentic systems. In *ICLR*, 2024. URL <https://arxiv.org/abs/2408.08435>.
- Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. AIDE: AI-driven exploration in the space of code, 2025. URL <https://arxiv.org/abs/2502.13138>.
- Li, A., Wu, C., Ge, Z., et al. The FM agent, 2025. URL <https://arxiv.org/abs/2510.26144>.
- Liu, S., Gao, C., and Li, Y. Large language model agent for hyper-parameter optimization. In *CPAL*, 2024. URL <https://arxiv.org/abs/2402.01881>.
- Liu, Z., Cai, Y., Zhu, X., et al. ML-master: Towards AI-for-AI via integration of exploration and reasoning, 2025. URL <https://arxiv.org/abs/2506.16499>.
- Nam, J., Yoon, J., Chen, J., Shin, J., Arik, S. O., and Pfister, T. MLE-STAR: Machine learning engineering agent via search and targeted refinement, 2025. URL <https://arxiv.org/abs/2506.15692>.
- Olson, R. S. and Moore, J. H. TPOT: A tree-based pipeline optimization tool for automating machine learning. In *Workshop on Automatic Machine Learning at ICML*, 2016.
- Sarch, G., Jang, L., Tarr, M. J., Cohen, W. W., Marino, K., and Fragkiadaki, K. VLM agents generate their own memories: Distilling experience into embodied programs of thought. In *NeurIPS*, 2024.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- Sutton, R. S., Precup, D., and Singh, S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112: 181–211, 1999.
- Talebirad, Y., Parsaee, A., Szepesvari, C. Y., Nadiri, A., and Zaiane, O. Toward a theory of hierarchical memory for language agents. In *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents)*, 2026.
- Thornton, C., Hutter, F., Hoos, H. H., and Leyton-Brown, K. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *KDD*, 2013.
- Toledo, E., Hambardzumyan, K., Josifoski, M., et al. AI research agents for machine learning: Search, exploration, and generalization in MLE-bench, 2025. URL <https://arxiv.org/abs/2507.02554>.
- Vezhnevets, A. S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., and Kavukcuoglu, K. Feudal networks for hierarchical reinforcement learning. In *ICML*, 2017. URL <https://arxiv.org/abs/1703.01161>.
- Wan, C., Dai, X., Wang, Z., Li, M., Wang, Y., Mao, Y., Lan, Y., and Xiao, Z. Loongflow: Directed evolutionary search via a cognitive plan-execute-summarize paradigm, 2025. URL <https://arxiv.org/abs/2512.24077>.
- Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *TMLR*, 2023. URL <https://arxiv.org/abs/2305.16291>.
- Weller, O., Boratko, M., Naim, I., and Lee, J. On the theoretical limitations of embedding-based retrieval. In *ICLR*, 2026. URL <https://arxiv.org/abs/2508.21038>.
- Xia, P., Chen, J., Wang, H., et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.08234>.
- Yang, X., Yang, X., Fang, S., et al. R&D-Agent: An LLM-agent framework towards autonomous data science, 2025. URL <https://arxiv.org/abs/2505.14738>.
- Zhang, L., Zhang, Y., Ren, K., Li, D., and Yang, Y. ML-Copilot: Unleashing the power of large language models in solving machine learning tasks. In *EACL*, 2024. URL <https://arxiv.org/abs/2304.14979>.
- Zhang, L. et al. AutoML-GPT: Automatic machine learning with GPT. *arXiv preprint*, 2023.
- Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., and Huang, G. Expel: Llm agents are experiential learners. In *AAAI*, 2023. URL <https://arxiv.org/abs/2308.10144>.
- Zhu, X., Chen, Y., Tian, H., et al. Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory, 2023. URL <https://arxiv.org/abs/2305.17144>.

A. Algorithms

A.1. Orchestrator

Algorithm 1 gives the top-level multi-competition loop described in §3.

Algorithm 1 HASTE orchestrator: multi-competition skill accumulation with LLM-driven promotion.

Require: Competitions $\mathcal{T}$, domain partition $\mathcal{D}$, initial skill store $\mathcal{S}$ (possibly $\emptyset$).

```

1: Tag each  $t \in \mathcal{T}$  with domain  $d(t) \in \mathcal{D}$ .
2: Pick one seed per domain; let  $\{R_1, R_2, \dots\}$  be the resulting rounds.
3: for round  $R_r$  in  $R_1, R_2, \dots$  do
4:    $\mathcal{L}_r \leftarrow \emptyset$  {new learnings this round}
5:   for competition  $t \in R_r$ , in parallel where possible do
6:      $\Lambda(t) \leftarrow \mathcal{S}^G \cup \mathcal{S}_{d(t)}^D \cup \mathcal{S}_t^C$  {tiered loading}
7:      $(\hat{y}_t, \ell_t) \leftarrow \text{SPECIALIST}(t, \Lambda(t))$  {Alg. 2}
8:      $\mathcal{S}_t^C \leftarrow \mathcal{S}_t^C \cup \ell_t$ ;  $\mathcal{L}_r \leftarrow \mathcal{L}_r \cup \ell_t$ 
9:   end for
10:  for learning  $\ell \in \mathcal{L}_r$  do
11:     $decision \leftarrow \text{PROMOTELLM}(\ell, \mathcal{S})$ 
12:    if  $decision = \text{GLOBAL}$  then
13:       $\mathcal{S}^G \leftarrow \mathcal{S}^G \cup \{\text{ABSTRACT}(\ell)\}$ 
14:    else if  $decision = \text{DOMAIN}$  then
15:       $\mathcal{S}_{d(t)}^D \leftarrow \mathcal{S}_{d(t)}^D \cup \{\text{ABSTRACT}(\ell)\}$ 
16:    else if  $decision = \text{CONFLICT}$  then
17:      keep both, annotate with conditions
18:    else
19:      leave in  $\mathcal{S}_t^C$  or skip
20:    end if
21:  end for
22: end for
23: return  $\{\hat{y}_t\}_{t \in \mathcal{T}}, \mathcal{S}$ 

```

A.2. Specialist Pipeline

Algorithm 2 formalizes the five-stage per-competition pipeline described in §3.2.

Algorithm 2 HASTE specialist: per-competition pipeline.

Require: Competition t , loaded skills $\Lambda(t)$.

```

1:  $profile \leftarrow \text{TASKPROFILER}(t)$  {metadata, CV strategy, resource probe}
2:  $\{m_1, m_2, m_3\} \leftarrow \text{PROTOTYPESCREEN}(t, profile, \Lambda(t))$ ;  $s_i \leftarrow \text{RUN}(m_i)$ 
3:  $win, ru \leftarrow$  top-2 prototypes by score
4:  $win^* \leftarrow \text{ADAPTIVEREFINE}(win, \Lambda(t), N=20)$  {Alg. 3}
5:  $ru^* \leftarrow \text{ADAPTIVEREFINE}(ru, \Lambda(t), N=6)$ 
6:  $ens \leftarrow \text{RANKAVERAGE}(\text{top-3 checkpoints from } win^* \cup ru^*)$ 
7:  $\hat{y} \leftarrow \arg \max_{c \in \{ens, win^*, ru^*\}} \text{score}(c)$  {accept ensemble only if it wins}
8:  $\ell \leftarrow \text{PRODUCELEARNINGS}(\text{history})$ 
9: return  $\hat{y}, \ell$ 

```

A.3. Adaptive Refinement Loop

Algorithm 3 gives the inner refinement loop invoked twice by the specialist, on lines 4 and 5 of Algorithm 2: once on the prototype winner with budget $N=20$, once on the runner-up with budget $N=6$. The three tiers, EXPLORING, OPTIMIZING, and FINETUNE, progress under auto-escalation. Two consecutive non-improvements advance the tier; stagnation at FINETUNE exits the loop; revert-on-regression keeps a bad change from carrying into the next step.

Algorithm 3 ADAPTIVEREFINE: linear refinement loop with auto-escalation and revert-on-regression.

```

1: procedure ADAPTIVEREFINE( $m, \Lambda, N$ )
2:    $tier \leftarrow$  EXPLORING;  $best \leftarrow m$ ;  $c \leftarrow 0$ 
3:   for  $i = 1$  to  $N$  do
4:      $p \leftarrow$  LLMPROPOSAL( $best, tier, \text{history}, \Lambda$ )
5:      $s \leftarrow$  RUN( $p$ )
6:     if  $s$  improves on score( $best$ ) then
7:        $best \leftarrow p$ ;  $c \leftarrow 0$ 
8:     else
9:       revert  $p$ ;  $c \leftarrow c + 1$ 
10:    end if
11:    if  $c \geq 2$  then
12:      if  $tier =$  FINETUNE then break                                ▷ stagnation exit
13:      else advance  $tier$ ;  $c \leftarrow 0$                                 ▷ auto-escalate
14:      end if
15:    end if
16:  end for
17:  return  $best$ 
18: end procedure

```

B. Prompts

We include the four key prompts that drive the system.

B.1. Prototype Screen

The prototype screen prompt (Figure 3) is used at the start of each competition. It injects the resource probe and any accumulated skills, and asks the LLM for three diverse model proposals.

Prototype Screen Prompt (abbreviated)

```
You are an ML engineer starting a Kaggle competition.
Competition: {competition_id}
Domain: {domain} Metric: {metric} ({direction})
Data: {rows} rows, {features} features
Target: {target_type}
[Resource budget: GPU model, VRAM, CPU cores, RAM]
[Accumulated skills from past competitions, if any]

Phase 0 is a diversity screen across model families --- the winner will be refined
later.

Propose 3 DIVERSE model approaches to try. Each should be a fundamentally different
strategy (e.g. gradient boosting vs. neural net vs. linear model, or different
preprocessing philosophies).

Return JSON:
{``models``: [{``name``: ..., ``description``: ...,
``change_specification``: ...}, ...]}
```

Figure 3. Abbreviated prototype screen prompt. The full prompt injects measured GPU/CPU/RAM from a resource probe and up to 2000 characters of accumulated skills. The LLM returns three model specifications, each executed with 1-fold validation on full data.

B.2. Refinement Proposal

The refinement proposal prompt (Figure 4) is the per-iteration call inside AdaptiveRefine. It carries the current tier, compressed history, and the loaded skills, and uses the six-mode failure taxonomy to structure the next change.

Refinement Proposal Prompt (abbreviated)

```
You are an ML engineer working on a Kaggle competition.
Competition: {competition_id}
Metric: {metric} ({direction})
Data: {rows} rows, {features} features
Current best score: {best_score}
Phase: {tier_label}
{tier_guidance}
[Resource budget, experiment history, accumulated skills,
current code (first 3000 chars)]

## Failure Mode Diagnosis
First, diagnose which failure mode best describes
the current state:
1. UNDERFITTING --- train score near baseline, small gap
2. OVERFITTING --- large train-val gap, high train score
3. FEATURE_GAP --- top features dominate, score plateaus
4. NOISE_CEILING --- high CV variance, score fluctuates
5. DISTRIBUTION_MISMATCH --- train-val-test disagreement
6. DIMINISHING_RETURNS --- each iter improves <0.1%
Mention which mode applies and which past skill (if any) influenced your decision.
Propose ONE atomic change to improve the score.
Return JSON: {'plan': ..., 'change_specification': ...,
'decision': 'CONTINUE' | 'NEXT_TIER' | 'STOP'}
```

Figure 4. Abbreviated refinement proposal prompt. The tier label rotates through Exploring, Optimizing, and Fine-tuning. The six-mode failure taxonomy gives the LLM structured diagnostic guidance. The `decision` field allows the LLM to self-escalate tiers or terminate early.

B.3. Learning Production

The learning production prompt (Figure 5) closes each competition. The specialist reflects on its full experiment history and emits two to five plain-text learnings, each with a proposed tier.

Learning Production Prompt (abbreviated)

```
You just finished working on ``{competition.id}``.
Domain: {domain} Metric: {metric} ({direction})
Dataset: {rows} rows, {features} features
## Your Experiments and Results
[Full experiment history with scores, deltas, kept/reverted status]

Write learnings that could help in future competitions.
Distinguish SUCCESS and FAILURE learnings:
- What worked --- ``this technique improved the score because...``
- What failed --- ``this technique hurt the score because...``
Failure learnings are equally valuable. Knowing what NOT to do saves future compute.
For failure learnings, note:
- WHY it failed (overfitting? wrong model family?)
- WHEN it might fail again (conditions to watch for)
Classify each learning:
- ``competition`` --- specific to this task only
- ``domain`` --- relevant to similar {domain} tasks
- ``global`` --- relevant to any ML task
Return JSON: {``learnings``: [{``title``: ...,
``body``: ..., ``proposedtier``: ...}, ...]}
Write 2--5 learnings. Focus on actionable, non-obvious insights.
```

Figure 5. The learning production prompt. Each specialist reflects on its full experiment history after completing a competition. Learnings are saved to the competition tier and later evaluated for promotion by the orchestrator.

B.4. Skill Promotion

The skill promotion prompt (Figure 6) runs between rounds. The orchestrator evaluates each new learning against existing skills and decides SKIP, COMPETITION, DOMAIN, GLOBAL, or CONFLICT.

Skill Promotion Prompt (abbreviated)

```
You are reviewing ML learnings produced by domain agents after completing competitions.
Your job is to decide which learnings should be promoted up the skill hierarchy.
[All existing global and domain skills, for deduplication]
[New learnings from all domain agents this round]

## Promotion Rules
For each learning, decide:
1. ``skip`` --- already covered or too obvious
2. ``competition`` --- too specific (dataset quirks, row indices)
3. ``domain`` --- generalizable to similar tasks. Abstract it.
4. ``global`` --- universally useful across all ML tasks
5. ``conflict`` --- contradicts an existing learning. Note conditions under which
each holds.

## Quality Standards
- Be selective. Promote AT MOST 50% of learnings.
- Abstractions MUST NOT mention specific competition
names, dataset names, or exact score values.
Bad: ``On aerial-cactus, AUC reached 0.9997``
Good: ``When AUC is near ceiling (>0.999),
further refinement yields diminishing returns``
- For ``conflict``: provide both the conflicting skill
ID and a condition annotation.
```

Figure 6. The skill promotion prompt. The orchestrator evaluates all new learnings against existing skills after each round. Abstractions strip competition-specific details to produce reusable domain or global knowledge.

C. Per-Competition Main Benchmark Results

Table 3 reports per-competition results on MLE-Bench Lite.

Table 3. Per-competition results on MLE-Bench Lite (22 competitions). Best score across all runs shown. — indicates the value was not recorded for that run.

Competition	Domain	Medal	Test Score	Phase0	Final CV	Hours	Iters
aerial cactus identification	Vision	Gold	1.00000	0.9994	1.0000	1.8	5
aptos2019 blindness detection	Vision	Gold	0.93723	0.9178	0.9280	11.5	1
denoising dirty documents	Vision	Gold	0.01127	0.0796	0.0122	10.5	6
histopathologic cancer detection	Vision	Gold	0.99692	—	—	12.0	—
plant pathology 2020 fgvc7	Vision	Gold	0.99637	—	—	12.0	—
the icml 2013 whale challenge ri...	Vision	Gold	0.99364	—	—	12.0	—
dogs vs cats redux kernels edition	Vision	Bronze	0.05514	—	—	12.0	—
siim isic melanoma classification	Vision	Bronze	0.93730	0.8890	0.9236	11.6	2
dog breed identification	Vision	—	0.58682	—	—	12.0	—
ranzcr clip catheter line classi...	Vision	—	0.95658	—	—	12.0	—
detecting insults in social comm...	NLP	Gold	0.95319	0.9026	0.9525	9.8	18
random acts of pizza	NLP	Silver	0.79847	0.7648	0.7916	1.8	13
spooky author identification	NLP	Silver	0.22115	0.3553	0.3040	12.0	11
jigsaw toxic comment classificat...	NLP	Bronze	0.98653	0.9621	0.9925	12.0	4
text normalization challenge eng...	NLP	Bronze	0.99038	—	—	12.0	—
text normalization challenge rus...	NLP	Bronze	0.97888	—	0.9791	10.5	10
nomad2018 predict transparent co...	Tabular	Gold	0.05470	0.0619	0.0533	6.6	11
tabular playground series dec 2021	Tabular	Gold	0.96300	0.9616	0.9621	10.6	1
leaf classification	Tabular	—	0.06652	0.5442	0.0498	2.9	11
new york city taxi fare prediction	Tabular	—	6.54783	3.5673	3.2401	11.8	8
tabular playground series may 2022	Tabular	—	0.99630	0.9461	0.9959	10.9	11
mlsp 2013 birds	Audio	Gold	0.96446	0.8094	0.9198	11.2	11

D. Cold Run vs. Warm Run

Table 4 compares cold-run and warm-run outcomes across all 22 MLE-Bench Lite competitions. Nine competitions already medaled on the cold run (with few or no accumulated skills) and were skipped in the rerun phase. Of the 13 that failed the cold run, 8 flipped to medal on the warm run with accumulated global and domain skills, accounting for the lift from 40.9% to 77.3% medal rate. The remaining 5 failed both attempts.

Table 4. Cold run vs. warm run across all 22 MLE-Bench Lite competitions. Cold = first attempt (few or no accumulated skills); Warm = later attempt with accumulated skills. Nine competitions already medaled cold and were not re-attempted. Of the 13 that failed cold, 8 flipped to medal on the warm run.

Competition	Cold Skills	Cold Medal	Warm Skills	Warm Medal	Outcome
<i>Already medaled on cold run (not re-attempted)</i>					
denoising-dirty-documents	5	G	—	—	already medaled
detecting-insults	5	G	—	—	already medaled
dogs-vs-cats	37	B	—	—	already medaled
histopathologic	14	G	—	—	already medaled
plant-pathology	14	G	—	—	already medaled
random-acts-of-pizza	5	S	—	—	already medaled
spooky-author	5	S	—	—	already medaled
tabular-playground-dec	21	G	—	—	already medaled
whale-challenge	14	G	—	—	already medaled
<i>Failed cold, flipped to medal on warm run</i>					
aerial-cactus	5	—	50	G	flipped
aptos2019	34	—	55	G	flipped
jigsaw-toxic	37	—	59	B	flipped
mlsp-2013-birds	32	—	60	G	flipped
nomad2018	14	—	60	G	flipped
siim-isic-melanoma	50	—	71	B	flipped
text-norm-english	50	—	65	B	flipped
text-norm-russian	50	—	72	B	flipped
<i>Failed both cold and warm runs</i>					
dog-breed	37	—	62	—	no medal
leaf-classification	29	—	54	—	no medal
nyc-taxi-fare	59	—	68	—	no medal
ranzcr-catheter	37	—	60	—	no medal
tabular-playground-may	21	—	60	—	no medal

E. Representative Skills

Table 5 shows representative entries from the skill hierarchy with their source competition and an estimate of how many refinement iterations they saved on later, similar tasks. The full skill inventory contains 5 global, 15 vision, 12 NLP, 19 tabular, and 108 competition entries.

Table 5. Representative skills from the 3-tier hierarchy. Each entry is a plain-text file traced to the experiment that produced it. Iteration savings are estimated from cold-start vs. skill-loaded runs of the same competition.

Tier	Type	Skill (abbreviated)	Source	Iters saved
Global	Tech.	Ensembling a strong model with a weaker one can degrade performance	dogs-vs-cats	~2 each
Global	Tech.	Larger architecture does not fix wrong problem formulation (ordinal vs. classification)	aptos2019	~1–2
Global	Tech.	Chance-level scores usually indicate a training bug, not a model limitation	detecting-insults	~1–2
Domain (NLP)	Tech.	DeBERTa-v3-base is the strongest starting point for text classification under 12h	detecting-insults	~2–3
Domain (Vis.)	Tech.	ConvNeXt-Large + 10-fold stratified CV is a strong default for small-dataset image classification	plant-pathology	~1–2
Domain (Tab.)	Tech.	Log-transforming the target can hurt RMSE on right-skewed regression targets	NYC-taxi-fare	~1
Domain (Vis.)	Refine	Vision refinement hints: 3/3 changes kept (100% hit rate). Try first: switch to ConvNeXt, add TTA	aerial-cactus	guided
Domain (NLP)	Refine	NLP refinement hints: 5/12 changes kept (42%). Try first: fix LR, add FGM adversarial training	detecting-insults	guided
Comp.	Tech.	Crystal descriptors (SOAP, Ewald, PRDF) cause regression on small crystal datasets (~5K samples)	nomad2018	~2

F. Ablation Resource Breakdown

Table 6 reports the full resource usage of the three ablation conditions: medal rate, mean test score, output token consumption, wall time, experiments attempted (with success rate), and skills loaded. The headline finding (flat doubles empty’s token cost for fewer medals) is in §4.5; this table provides the supporting detail.

Table 6. Resource usage under three skill-loading conditions. Flat uses $2\times$ the output tokens of Empty without improving medal rate.

Metric	Tiered	Flat	Empty
Medal rate	100% (8/8)	62.5% (5/8)	62.5% (5/8)
Mean test score	0.949	0.910	0.893
Total output tokens	2.27M	3.78M	1.86M
Mean output tokens	284K	472K	232K
Tokens per medal	284K	756K	372K
Mean wall time (h)	10.2	10.5	10.7
Total experiments	60	75	65
Skills loaded (mean)	36.9	159	0
Context from skills	~25K chars	~145K chars	0

G. Ablation Per-Competition Table

Table 7 gives the full per-competition scores and medals for the controlled ablation (Figure 2 in the main text).

Table 7. Controlled ablation: scores and medals (G/S/B/—) under three skill-loading conditions (8 competitions, 1 seed). **Tiered**: domain-scoped; **Flat**: all 159 skills; **Empty**: none. Best score per competition in **bold**.

Competition	Domain	Tiered	Flat	Empty
detecting-insults	NLP	0.953 G	0.958 G	0.956 G
jigsaw-toxic	NLP	0.987 B	0.981 —	0.985 —
random-acts-of-pizza	NLP	0.798 S	0.599 —	0.481 —
histopathologic	Vision	0.997 G	0.998 G	0.998 G
plant-pathology	Vision	0.996 G	0.999 G	0.994 G
aptos2019	Vision	0.937 G	0.921 S	0.934 G
tabular-playground	Tabular	0.963 G	0.963 G	0.962 G
mlsp-2013-birds	Audio	0.964 G	0.860 —	0.832 —
Medal rate		100% (8/8)	62.5% (5/8)	62.5% (5/8)
Mean score		0.949	0.910	0.893